



Set Up Kubernetes Deployment

N

Nabil Ahmed

```
[+] Building 12.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> => sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> => sha256:a381ce006821bdce2a0d62e601890f912e9f23d9e1fb61007e58328863bd7708 1.73kB / 1.73kB
=> => sha256:ac5fec9ec2244c78ecbb822308adf80f8429fc6aalef08f75572ffdb2124470d 5.18kB / 5.18kB
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfalb 3.80MB / 3.80MB
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfalb
=> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d
=> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3
=> [internal] load build context
=> => transferring context: 42.31kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt requirements.txt
=> [4/5] RUN pip3 install -r requirements.txt
=> [5/5] COPY . .
=> => exporting to image
=> => exporting layers
=> => writing image sha256:974ccf32bfb9d8f710ed6a63c77e468ceff4eb8342aeda8497b43afd344b68c8
=> => naming to docker.io/library/nextwork-flask-backend
```



Introducing Today's Project!

In this project, I will prepare a backend app for Kubernetes deployment. This is because Kubernetes deployment requires apps to be containerised and for their to be a container image. This project will get me to create that container image.

Tools and concepts

I used Amazon EKS, Git, Docker and Amazon ECR to prepare a backend app for deployment with Kubernetes. Key steps include setting up an EKS cluster, cloning code from GitHub, building and pushing docker image to Amazon ECR.

Project reflection

This project took me approximately. 1 hour 30 minutes. The most challenging part was running into the multiple errors. My favourite part was combining topics I learn previosuly, such as Docker, to see an image in my ECR repository.

Something new that I learnt from this experience was needing to refresh the instance connection session so that the ec2-user could obtain the permission to run docker commands after adding it to the docker group.



What I'm deploying

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included launching an EC2 instance, installing eksctl, and modifying the IAM role for my instance so that it has AdministratorAccess. I also ran an eksctl command.

I'm deploying an app's backend

Next, I retrieved the backend that I plan to deploy. An app's backend means the logic that defines how the app 'works'. I retrieved backend code by cloning it from a GitHub repository.

```
[ec2-user@ip-172-31-9-108 ~]$ git clone https://github.com/NatNextWork1/nextwork-flask-backend.git
Cloning into 'nextwork-flask-backend'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
Receiving objects: 100% (18/18), 6.14 KiB | 6.14 MiB/s, done.
remote: Total 18 (delta 4), reused 1 (delta 0), pack-reused 0 (from 0)
Resolving deltas: 100% (4/4), done.
[ec2-user@ip-172-31-9-108 ~]$ ls
nextwork-flask-backend
```



Building a container image

Once I cloned the backend code, my next step is to build a container image of the backend. This is because Kubernetes needs a container image for a successful app deployment. At the moment I have not prepared a container image yet (just raw code).

When I tried to build a Docker image of the backend, I ran into a permissions error because I am logged into my EC2 instance as the ec2-user (created automatically from the EC2 instance's AMI). ec2-user cannot run root user level commands without 'sudo'.

To solve the permissions error, I added ec2-user to the Docker group. The Docker group is a group in Linux systems that grants a user the permissions to run Docker commands (like docker build).

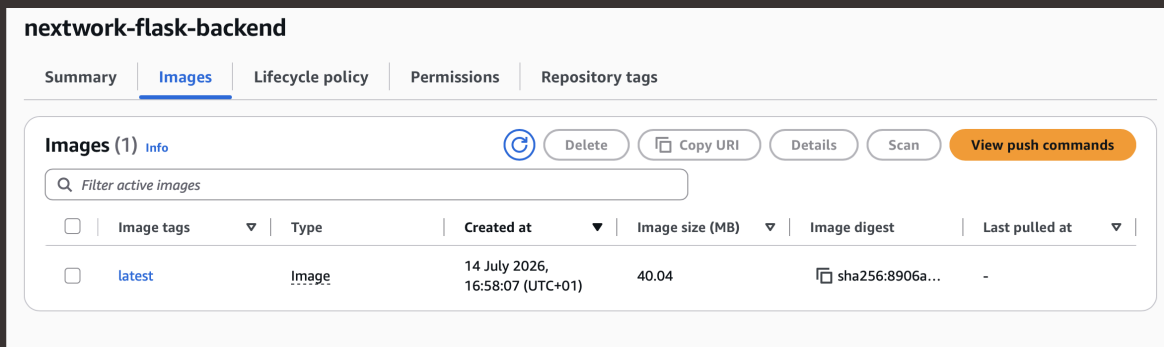
```
[+] Building 12.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> => sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> => sha256:a381ce006821bdce2a0d62e601890f912e9f23d9e1fb61007e58328863bd7708 1.73kB / 1.73kB
=> => sha256:ac5fec9ec2244c78ecbb822308adf80f8429fc6aa1ef08f75572ffdb2124470d 5.18kB / 5.18kB
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b
=> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d
=> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3
=> [internal] load build context
=> => transferring context: 42.31kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt requirements.txt
=> [4/5] RUN pip3 install -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> => writing image sha256:974ccf32bfb9d8f710ed6a63c77e468ceff4eb8342aeda8497b43afd344b68c8
=> => naming to docker.io/library/nextwork-flask-backend
```



Container Registry

I'm using Amazon ECR in this project to store the container image. ECR is a good choice for the job because it is an AWS service and integrates well with other AWS services like EKS. This makes deploying the app faster.

Container registries like Amazon ECR are great for Kubernetes deployment because I get to store tagged images from a single source truth. This means that users/other services can pull my latest image without any manual downloading required.





nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

